MEMORIA DE LA PRÁCTICA

MF0493_3 · Implantación de aplicaciones web

Aplicación web de gestión de biblioteca con Spring MVC, Hibernate y Maven

Juan José García Geijo · 27 de octubre de 2026

1. Datos personales

Campo	Dato
Nombre y apellidos	Juan José García Geijo
DNI / NIE	(completar)
Módulo	MF0493_3 · Implantación de aplicaciones web
Práctica	Aplicación web de gestión de una entidad (biblioteca)
Repositorio	https://github.com/Noes3no/Ejercicio_jj
Fecha de entrega	27 de octubre de 2026

Nota: sustituye «(completar)» por tu DNI o NIE antes de exportar el PDF definitivo.

2. Descripción de la aplicación

La aplicación es un gestor de biblioteca accesible desde el navegador. Muestra el catálogo de libros almacenados en una base de datos, permite dar de alta nuevos títulos mediante un formulario y eliminar los registros existentes. Toda la información se guarda mediante Hibernate en una base de datos H2 en memoria, por lo que los datos de ejemplo se cargan al arrancar y se pierden al detener la aplicación.

La entidad elegida es el libro, con tres campos: identificador, título y autor. La aplicación sigue el patrón MVC de Spring: el controlador atiende las peticiones HTTP, el repositorio accede a los datos y la vista Thymeleaf presenta la información al usuario.

2.1 Funcionalidades

- **Listado:** Tabla con todos los libros guardados (identificador, título, autor y acción de borrado).
- **Alta:** Formulario con validación HTML de campos obligatorios que inserta un nuevo libro en la base de datos.
- **Borrado:** Enlace «Borrar» en cada fila que elimina el registro indicado y vuelve al listado.
- **Persistencia automática:** Hibernate genera la tabla a partir de la clase @Entity y ejecuta las consultas SQL necesarias.
- **Datos de ejemplo:** El archivo data.sql carga tres libros al iniciar la aplicación.

2.2 Tecnologías utilizadas

Tecnología	Versión	Uso en el proyecto
Java	17	Lenguaje de programación
Spring Boot	3.3.5	Arranque y configuración automática
Spring MVC	Incluido en Spring Boot	Controladores y vistas
Spring Data JPA / Hibernate	Incluido en Spring Boot	Persistencia objeto-relacional
H2	Incluido en Spring Boot	Base de datos en memoria
Thymeleaf	Incluido en Spring Boot	Plantillas HTML
Maven	Incluido en el kit	Dependencias y empaquetado
JUnit 5	Incluido en Spring Boot	Pruebas automáticas

3. Arquitectura y estructura del proyecto

El flujo de una petición es el siguiente: el navegador solicita /libros, el controlador pide la lista al repositorio, Hibernate traduce la petición a SQL, la base de datos devuelve las filas, Hibernate las convierte en objetos y Thymeleaf genera la tabla HTML que recibe el navegador.

Archivo	Función
Application.java	Punto de entrada de Spring Boot
Libro.java	Entidad JPA: se convierte en la tabla libro
LibroRepository.java	Repositorio que extiende JpaRepository (findAll, save, deleteById)
LibroController.java	Controlador con las rutas de listado, alta y borrado
lista.html	Vista Thymeleaf con la tabla y el formulario
application.properties	Configuración de la base de datos, JPA, puerto y codificación
data.sql	Datos de ejemplo
src/test	Pruebas automáticas de estructura y funcionales

```
src/main/java/es/hazerta/app/Application.java
src/main/java/es/hazerta/app/Libro.java
src/main/java/es/hazerta/app/LibroRepository.java
src/main/java/es/hazerta/app/LibroController.java
src/main/resources/application.properties
src/main/resources/data.sql
src/main/resources/templates/lista.html
src/test/java/es/hazerta/app/EstructuraTest.java
src/test/java/es/hazerta/app/FuncionalTest.java
```

En esta práctica la lógica de negocio es mínima, por lo que el controlador utiliza directamente el repositorio. Si se amplía el proyecto, lo recomendable es intercalar una capa Service entre el controlador y el repositorio, tal como pide el pliego del módulo.

4. Diagrama E-R

El modelo implementado consta de una única entidad, LIBRO, con su clave primaria y dos atributos descriptivos. Hibernate crea la tabla automáticamente a partir de las anotaciones de la clase.

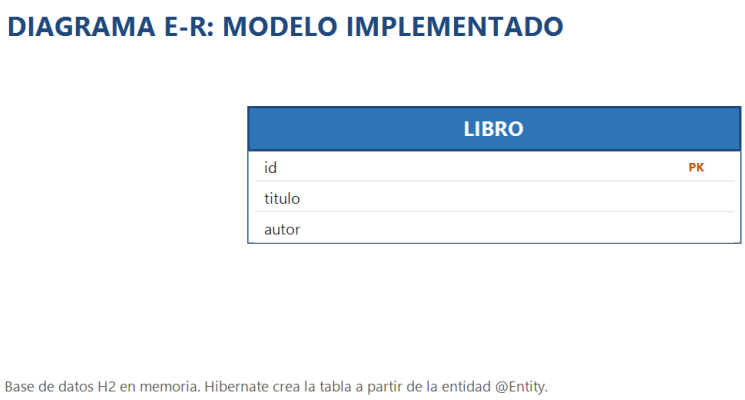


Figura 1. Diagrama E-R del modelo implementado.

El pliego del módulo pide un mínimo de dos tablas relacionadas 1:N. Como mejora, se propone añadir la entidad PRESTAMO, relacionada con LIBRO mediante una clave ajena: un libro puede tener muchos préstamos y cada préstamo pertenece a un único libro.

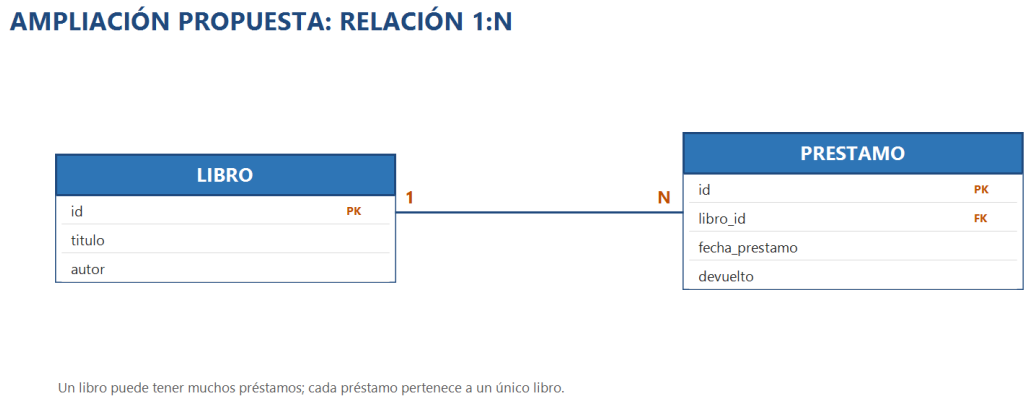


Figura 2. Ampliación propuesta con la relación 1:N entre LIBRO y PRESTAMO.

5. Funcionamiento de la aplicación

La aplicación se arranca con arrancar.bat o con java -jar app.jar y se abre en el navegador la dirección http://localhost:8080, que redirige al listado de libros.

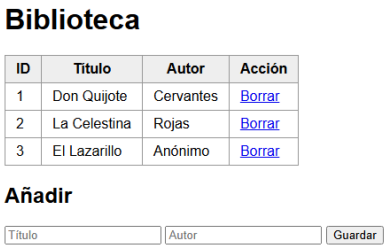


Figura 3. Pantalla principal con el listado de los tres libros de ejemplo y el formulario de alta.

Al rellenar el formulario y pulsar Guardar, el controlador recibe los datos por POST, el repositorio guarda el nuevo libro y el navegador vuelve al listado mostrando el registro añadido.

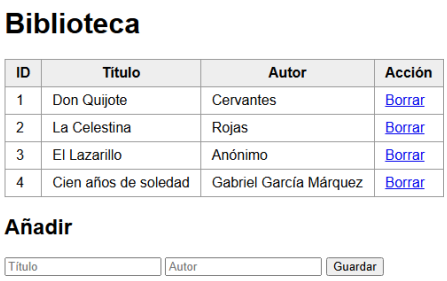


Figura 4. Listado tras dar de alta el libro «Cien años de soledad».

El enlace Borrar de cada fila envía una petición GET a /libros/borrar/{id}, el controlador elimina el registro con deleteById y se vuelve al listado actualizado.

6. Despliegue en otro entorno

6.1 Requisitos

- **Java:** Java 17 o superior (JRE o JDK). No hace falta instalar Maven, Tomcat ni la base de datos.
- **Sistema operativo:** Windows, Linux o macOS con conexión a un navegador actual.
- **Puerto libre:** El 8080 debe estar disponible, o bien se indica otro puerto al arrancar.

6.2 Pasos de despliegue

1. Generar el paquete ejecutable desde la carpeta del proyecto. Se puede hacer con el script incluido o con Maven:

```
empaquetar.bat

# equivalente con Maven
mvn -o clean package -DskipTests
```

2. Copiar el archivo target/app.jar al equipo de destino. El JAR contiene la aplicación y todas sus dependencias.

3. Ejecutarlo desde una consola con Java 17 o superior:

```
java -jar app.jar
```

4. Abrir el navegador en la dirección del listado:

```
http://localhost:8080/libros
```

5. Para detener la aplicación, pulsar Ctrl + C en la consola.

Si el puerto 8080 está ocupado, se puede arrancar en otro puerto sin modificar el proyecto:

```
java -jar app.jar --server.port=9090
```

6.3 Despliegue con el proyecto completo

Si se dispone del código fuente y del kit de herramientas, dentro de la carpeta del proyecto se pueden usar los scripts:

- **probar.bat:** Ejecuta las pruebas automáticas.
- **arrancar.bat:** Arranca la aplicación con el servidor embebido.
- **empaquetar.bat:** Genera target/app.jar.
- **ejecutar_jar.bat:** Ejecuta el JAR ya generado.

6.4 Comprobación

Tras el despliegue hay que verificar que el listado muestra los tres libros de ejemplo, que el alta añade una fila nueva y que el enlace Borrar la elimina. La base de datos H2 está en memoria: al reiniciar la aplicación vuelven a aparecer únicamente los datos de data.sql.

7. Pruebas realizadas

El proyecto incluye ocho pruebas automáticas que se ejecutan con probar.bat o con mvn test. Comprueban tanto la estructura del código como el funcionamiento real de las páginas.

Prueba	Qué comprueba	Resultado
todo1_entidad	La clase Libro lleva @Entity	Correcta
todo2_id	El campo id lleva @Id	Correcta
todo3_repositorio	LibroRepository extiende JpaRepository	Correcta
todo4_controlador	LibroController lleva @Controller	Correcta
arranca_aplicacion	La aplicación arranca y hay 3 registros de ejemplo	Correcta
todo5y8_listar	La página /libros muestra los datos en una tabla	Correcta
todo6_guardar	El alta añade una fila	Correcta
todo7_borrar	El borrado elimina una fila	Correcta

Resultado de la ejecución:

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Además se comprobó el despliegue real con java -jar app.jar: la página respondió con código 200, se añadió un libro desde el formulario y se eliminó correctamente.

8. Conclusiones y mejoras

La aplicación cumple el objetivo de la práctica: gestión de una entidad con acceso a base de datos mediante Hibernate y Spring MVC, con vistas de listado, alta y borrado, validación básica del formulario, proyecto Maven, pruebas automáticas y JAR desplegable.

- **Limitaciones:** El modelo tiene una sola tabla, no incluye edición de registros ni capa Service, y la validación se apoya en los atributos required del formulario.
- **Mejora 1:** Añadir la entidad PRESTAMO con relación @OneToMany / @ManyToOne y las vistas de préstamos.

- **Mejora 2:** Incorporar una capa Service, edición de libros y validación con Bean Validation (@NotBlank, @Size).
- **Mejora 3:** Sustituir H2 en memoria por una base de datos persistente (H2 en fichero o MySQL) y añadir un formulario de búsqueda.
- **Incidencia corregida:** Los acentos de data.sql aparecían corruptos en pantalla porque Spring leía el script con la codificación del sistema. Se corrigió añadiendo spring.sql.init.encoding=UTF-8 en application.properties.

9. Enlaces

Repositorio del proyecto: https://github.com/Noes3no/Ejercicio_jj

Documentación complementaria: ANALISIS_DE_ERRORES.docx y REGISTRO_DE_CAMBIOS_Y_SOLUCIONES.docx, incluidos en la carpeta 0_DOC_SOLUCIONES Y CAMBIOS.